

Long-term Blockchain Transactions Spanning Multiplicity of Smart Contract Methods

Chris G. Liu¹, Peter Bodorik¹ and Dawn Jutla²

¹ Faculty of Computer Science, Dalhousie University, Halifax, Nova Scotia B3H 4R2, Canada

² Saint Mary's University, Halifax, Nova Scotia B3H 3C3, Canada

Chris.Liu@dal.ca, Peter.Bodorik@dal.ca, Dawn.Jutla@gmail.com

Abstract. Research on blockchains addresses multiple issues, with one being writing smart contracts. In developing smart contracts, blockchains have complex constraints that complicate the design and development of smart contracts, software that controls the blockchain transactions. Blockchain applications that involve collaborations of multiplicity of actors, such as in trade of goods and services, give rise to additional complexity in that transaction activities tend to be long term, perhaps hours or days, and thus spanning multiplicity of methods of a smart contract. The problem is that a block-chain only supports the concept of a transaction within the scope of an execution of a single smart contract method. We compare the requirements and properties of long-term transactions and blockchain transactions and examine their mismatch. We then propose how to represent long-term blockchain transactions in smart contracts, and we describe a mechanism to support such transactions using patterns. Finally, we evaluate the proposed mechanism in terms of supporting transactional properties and its cost of execution.

Keywords: Blockchain; Long-term Blockchain Transaction; Smart Contract Transaction Spanning Multiplicity of Methods; Transaction Mechanism; Optimistic Commitment Methods

1 Introduction

Blockchain scalability, transaction throughput, and high cost are some of the main issues investigated by research on the blockchain infrastructure. Smart contract design and development also received much attention by the research and development communities as is evidence by numerous surveys, e.g., [1]-[3]. Khan et al. [1] survey and classify research on blockchain smart contracts into two broad categories. The usage category, which contains a large body of papers that explore usage of the blockchain technologies in many different and dissimilar fields, such as IoT, health, voting, supply chains, and the list goes on. The second category includes research on improvement of smart contracts by developing approaches and tools to improve smart contracts either when writing them or once they exist.

A sub-category of smart-contract improvement includes approaches that define a programming language that facilitates creation of smart contracts with desirable

properties. Although such approaches lead to smart contract programs with formally proven safety properties, they have not been adopted in practice by developers due to their perceived complexity. In another sub-category includes research that uses a well-known and used Business Process Management Notation (BPMN) to model the application requirements and transform a BPMN model automatically into the methods of a smart contract(s) [6]-[10]. In some instances, the BPMN model is augmented with additional information that is required for the transformation process from a BPMN model into methods of a smart contract. The additional information deals primarily with details on information/data as it flows through the PBMN model [9].

1.1 Motivation

Trade of goods and services, as opposed to securities, includes many types of collaborative activities that arise with a sale or trade of goods and/or services between two or more parties. Purchase of goods or services, particularly for higher priced items such as combines, may include many types of different activities, such as price negotiations, letter-of-credit financial instrument to guarantee payment, and transport of goods that may also include insurance and customs. Thus, trade transactions involve many parties that collaborate in activities that may be long term, e.g., spanning, minutes, hours, or days, as the collaborative activities may involve different software or human events that trigger invocation of the smart contract methods by a distributed application. Furthermore, activities may be dependent on each other in that an activity may have to wait until output is produced by another activity, output that affects how the activity is completed. We shall refer to such transactions simply as *long-term collaborative transactions*, or as *long-term transactions*.

Blockchains are not yet ready to support general transactions that appear in trade of goods and services. The difficulties arise due to the mismatch between the blockchain transaction definition and the long-term transactions requirements that are the results of collaborative activities. The overall result is that a transaction for trade of goods and services is naturally expressed as a collaboration of a multiplicity of methods, wherein a blockchain only supports a transaction that is a result of any one method of a smart contract that writes to the ledger. This results in a *mismatch between the long-term transaction requirements and transactions properties of current blockchains*. This mismatch is analogous to the *object relational impedance mismatch* [11]. And how to overcome this mismatch is the main goal of this paper.

1.2 Goal and Objectives

The goal of this paper is to propose an approach to overcoming the blockchain limitation and propose an approach to supporting a long-term transaction that is a result of execution of a multiplicity of methods of a smart contract – and, currently, a blockchain transaction is the result of an execution of an execution of any (one) smart contract method that writes to the ledger. Objectives of this paper are:

- Representation of a long-term transaction in a smart contract.
- Supporting mechanism that implements a long-term blockchain transaction that spans a multiplicity of executions of smart contract methods.
- Minimization of the developer's effort in declaring a long-term transaction and in implementing the long-term transaction mechanism.
- Evaluation of the proposed approach to supporting long-term transactions in terms of the transactional properties and overhead costs.

1.3 Contributions

Meeting the above objectives, we provide the following contributions to the problem of supporting long-term transactions, i.e., transactions that span a multiplicity of executions of smart contract methods:

- **Long-term Blockchain (LtB) transaction representation:** We present motivation for support of a long-term blockchain transaction and propose how to represent/define an LtB transaction in current blockchains.
- We analyze the source of the mismatch between the requirements of LtB transaction mechanisms and the properties of transactions of current blockchains and propose an approach for the LtB mechanism implementation using patterns.
- Our approach for the proposed LtB transaction mechanisms minimizes the developer's involvement in facilitating the LtB transaction mechanism: The developer simply creates the smart contract that contains the LtB transaction definition and then invokes a tool that amends the smart contract methods with appropriate patterns that implement the mechanism to support LtB transactions.
- We evaluate our transactional mechanism in terms of the transactional properties they support and overhead cost.

1.4 Outline

The second section describes how to represent a long-term blockchain transaction and provides an overview of our approach with an objective to minimize the software developer's effort in writing the long-term transactions and in supporting them, i.e., in implementing a mechanism to support the required transactional properties. The third section describes our approach to implementing the long-term transaction mechanism by augmenting the smart contract written by the developer with patterns that implement the long-term transaction mechanism. The fourth section evaluates the overhead cost of the mechanism. The penultimate section provides related work, while the last section provides a summary and concluding remarks, and our future research.

2 Long-term Blockchain Transaction

Blockchain transactions and long-term transaction requirements are first discussed and compared to identify shortcomings of current blockchain transactions in supporting the requirements of a long-term transaction. We then propose how a long-

term transaction may be represented and discuss properties that should be supported by a mechanism to support such transactions.

We then identify general approaches to supporting such transactions and select the approach we deem the most appropriate. We then propose a definition of a long-term blockchain transaction and discuss the long-term transaction properties.

2.1 Blockchain Transaction vs. Long-term Transaction

As blockchains are frequently referred to as distributed DBs with additional features, and the concept of DB transactions is well established and understood, we first briefly review DB transactions and their properties and then review how such properties are supported by the blockchain transaction mechanism. We then compare the blockchain and long-term transactions to derive the transactional properties that need to be supported by mechanism to support the long-term blockchain transactions.

DB transactions. A DB transaction is a collection of reads and writes issued against a DB, wherein the reads and writes are delineated explicitly by the application using the transaction begin and end operations/instructions. The DB system ensures that the DB operations within a transaction satisfy the properties of atomicity (all or nothing), consistency (correctness when executing transactions concurrently), isolation (no dirty reads), and durability. These properties are referred to briefly as transactional or ACID (Atomicity, Consistency, Isolation, Durability) properties. The operations issued against the DB may simply be referred to as Create, Read, Update, Delete (CRUD) operations.

Blockchain transactions. Documentation on the various blockchains generally states that a transaction is any change to the state of the ledger. Thus, a blockchain transaction may be defined in the following way:

***Definition:** A blockchain transaction is a collection of reads and appends to the (immutable and append-only) ledger made by an execution of any of the smart contract methods that writes to the ledger.*

Thus, the scope of a transaction is a sequence of reads and appends, also referred to as writes, to the ledger as executed by operations of any one of the smart contract methods that updates the ledger.

In a blockchain, ledger writes made by an execution of a smart contract method are submitted for inclusion in a block that is appended to the blockchain. Block validation ensures that updates made by a smart contract method are consistent relative to the state of the ledger and relative to all transactions within that block. Consequently, writes of a smart contract method form an *atomic* transaction in that either all writes (appends) made by the method are performed on the ledger or none are. Furthermore, the consensus method also ensures consistency because all ledger updates included in a block are validated to be consistent relative to the ledger and relative to the other transactions within the same block. As smart contract methods may access the transaction's updates only after the block is appended to the blockchain, dirty reads

are prevented and hence *isolation* is also achieved. Finally, as updates are written to the replicated ledger, the successfully completed/committed transactions are *durable* in that the results of their executions are not lost after the block is appended to the chain.

In short, the ACID transactional properties are supported by the current blockchain transaction and hence also need to be supported by any mechanism supporting the long-term transactions.

Long-term transactions and their properties. We already described the need for a long-term transaction, perhaps spanning days when representing transportation of a product. Such activities are not naturally represented by one blockchain method, but rather as a collection of methods of a smart contract. Thus, a *long-term transaction in a smart contract is a subset of the smart contract methods that read and write to the ledger such that ACID properties are supported*. In the following subsections we propose how to represent a long-term transaction in a smart contract.

2.2 Representing/Defining an LtB Transaction

LtB transaction scope. The scope of the transaction is determined by the developer by identifying which of the methods of the smart contract form the transaction, and it thus includes all ledger data accessed by those transaction methods. Thus, any ledger writes made by the transaction methods must be accessible to all the transaction methods before the transaction is committed. In other words, all ledger writes made by the transaction methods *must persist across executions of the transaction methods before the transaction is committed*.

Furthermore, as the state of a transaction should not be visible outside the scope of the transaction and should be accessible only by the transaction methods, the transaction methods must be *independent*, i.e., they may not access any of the non-transaction methods of the smart contract.

Another question that arises is how to identify the transaction begin and end and the scope of the transaction. We discuss how we approach these two issues and then present our proposal for defining a LtB transaction for smart contracts.

Identifying the LtB transaction start and end. Generally, identifying the start and an end of a transaction can be achieved in two ways:

1. As in a DB system, the application program signals explicitly the start and end of a transaction by invoking two special methods dedicated to such purposes.
2. There is no explicit signalling to begin or end a transaction. Instead, the mechanism itself determines when a transaction begins and ends by keeping track of the internal state of the transaction.

We choose the first method of explicit signalling using two methods of the long-term transaction mechanism, one method to begin the transaction and one to end it. The reasons for selecting this option are:

- This option is analogous to DB systems in which an application signals explicitly to the DB system the beginning and ending of a transaction.

- Perhaps the main reason is that an end of a transaction, whether successful or not, requires that certain activities be completed to terminate the transactions. Ensuring that the transaction termination activities are properly handled is easier if they are situated in one method, the method dedicated for ending the transaction.

Long-term Blockchain (LtB) transaction definition.

Definition: Long-term Blockchain (LtB) transaction in a smart contract is a subset of the smart contract methods, such that the transaction methods are *independent* and include two special methods, *LtB-transaction-begin* and *LtB-transaction-end*, to start and end the transaction, respectively.

The semantics of the long-term transaction model are:

- The LtB transaction mechanism enforces the ACID transactional properties.
- Methods of an LtB transaction must be *independent* in that they may not access any objects or data that are external to the LtB transaction.
- *LtB-transaction-begin* method must be invoked by the application to begin the transaction.
- *LtB-transaction-end* method is invoked by the application to terminate the transaction, wherein:
 - The first input parameter indicates whether the transaction completion is commitment or abort of the transaction.
 - The method may return a value that is defined by the developer.

2.3 LtB Transaction Mechanism Implementation Using Patterns

The LtB transaction mechanism may be created using two general approaches:

1. Augment the blockchain infrastructure to support a blockchain transaction that spans execution of more than one method.
2. Prepare software patterns as an add-on mechanism that utilizes the current blockchain infrastructure but provides support for LtB transaction.

The first option would be a major undertaking that requires major changes to the blockchain infrastructure and thus requiring major redesign, resources, and time. The second option is viable in our context, provided that the proposed mechanism to support blockchain trade transactions is (relatively) easy to use by a developer when writing a blockchain smart contract. We adopt the second option as it is less disruptive.

Developer's responsibilities in implementing the LtB transaction. There are two major issues that need to be addressed when considering using patterns for implementing the LtB transaction mechanism:

- The first one deals with how a *long-term transaction is defined/represented* while at the same time minimizing the developer's effort, which is addressed in a following subsection.

- The second one is on finding a mechanism to support LtB transactions and their properties, which is the topic of the next section.

From the implementation perspective, the developer can *apply patterns* on smart contracts, patterns such as those that facilitate access control [12] or patterns that implement the LtB transaction mechanism, which is our objective. Applying the patterns means that once the developer prepares the LtB transaction methods, appropriate patterns must be inserted in appropriate places in the smart contract methods for the patterns to implement the LtB transactions with their properties. However, instead of relying on the developer to insert such patterns in the LtB transaction code, to minimize the developer's effort and room for errors, the pattern augmentation should be automatically provided by a tool, such as a pre-compiler:

1. The developer writes the methods of the smart contract and methods of a long-term transaction as per usual but with additional requirements:
 - (a) The transaction methods must be *independent* in that they do not make any reference to object or methods that are not defined within the scope of the transaction.
 - (b) The developer must provide the pre-compiler with information on which of the smart contract methods form the *LtB transaction methods*.
2. The developer uses a pre-compiler to augment the transaction methods with appropriate patterns at appropriate places in the smart contract methods.

The transaction must be started and ended by invocation of the *LtB-transaction-start* and *LtB-transaction-end* methods, respectively, that must be invoked by the application to signal the start and end of a transaction. However, the two methods are not provided by the developer, but rather automatically through pattern augmentation.

The developer must identify, using special codes, which of the smart-contract methods belong to the LtB transaction. This is required so that prior to the compilation, the smart contract methods can be augmented with patterns that implement an LtB transaction mechanism using a pre-compiler that recognizes the special codes identifying the LtB transaction methods. We could create a tool to perform the pattern augmentation and provide it with information on which of the methods form the LtB transaction; however, once the tool is invoked, its functionality would be equivalent to that of the pre-compiler.

The smart contract methods are augmented with appropriate patterns that implement a LtB transaction mechanism, while ensuring that the transaction state persists across the execution of an LtB transaction. Only after the LtB transaction patterns are augmented can the smart contract be compiled and deployed. Information provided to the pre-compiler is the *minimal information* needed by any mechanism used to provide an LtB transaction mechanism.

3 LTB TRANSACTION MECHANISMS

First, we review how the blockchain properties affect the design of an LtB transaction mechanism and then we present the LtB transaction mechanism architecture.

3.1 Impact of Blockchain's Immutability on the LtB Transaction Mechanism

Blockchain ledger is immutable by allowing only read and append operations, wherein appends are also referred to as writes. To ease the development of smart contracts, the developer views the ledger content as a key-value store, wherein the blockchain infrastructure looks after creating a new instance of the object written to the DB by issuing a *write* (k, v) operation in which k is the key and v is the value to be written.

Implications of the above are that, if an LtB transaction method issues a ledger append/write, when the method issuing such a write completes, a new instance is created and that new instance is also available and can be read by any smart contract methods if the method knows the value of the key, k . However, an LtB transaction contains executions of a multiplicity of the transaction methods and, consequently, any ledger writes by one of the transaction methods is immediately available for reading by any of the smart contract methods. More importantly, as the ledger is immutable, the LtB transaction mechanism cannot “undo” a transaction’s write to the ledger by restoring the item’s old image/value before the update, as may be performed in a DB. Thus, any mechanism supporting the ACID properties can use only *optimistic* concurrency control approaches, in which the transaction’s writes to the ledger are made in some *private workspace*. Only when the transaction is committed are its ledger writes made to the ledger itself as a part of the commitment phase of the transaction.

The main issue to overcome is that the LtB transaction mechanism can only use the native blockchain transaction support that provides for a transaction as reads and writes made by an execution of only one smart-contract method. Thus, the LtB transaction mechanism must record ledger reads and writes in some *private workspace*, to which we shall refer as an *LtB transaction cache*, or simply as a *cache* for short. Only when the *LtB-transaction-end* method is called, that method needs to propagate all reads and writes, performed on the ledger data in the cache, from the cache to the ledger itself as a part of execution of that method.

3.2 Private Workspace

The pre-compiler augments the smart contract methods with the following patterns.

- The method *LtB-transaction-begin* is created as a part of an LtB transaction, wherein the method prepares the cache and seeds it with the ledger initial data.
- The cache must persist across executions of the LtB transaction methods and must be accessible by them.

- Each read or write to the ledger by any of the LtB transaction methods must be replaced with a pattern to make that read or write using the cache instead.
- The method *LtB-transaction-end* is created using patterns to propagate all transaction reads and writes, made using the cache, to the ledger itself.

It should be noted that the cache stores all ledger reads and writes made by the LtB transaction methods. Furthermore, the *LtB-transaction-end* method does not only propagate/replay all writes from cache to the ledger itself, but it also replays reads, i.e., it re-issues all reads that the LtB transactions made on the ledger. This is required, as some blockchains, such as Hyperledger fabric, need to know both the read and write sets of a blockchain transaction to ensure consistency/correctness of the transactions.

Also to be noted is that, due to the dynamic nature of the LtB transaction methods, they may perform reads on the cache that cause cache faults that need to be serviced from the ledger, i.e., the LtB transaction must be augmented by a method in the smart contract that reads the ledger and stores the read values in the cache and thus service the cache fault.

3.3 LtB Transaction Mechanism

The following subsection presents an architecture for cache implementation that exploits data structures, which are provided by some of the blockchains, that persist across execution of smart contract methods. Following this we describe the LtB transaction mechanism, while in the next section we perform its evaluation.

Cache Hosted by Persisting Blockchain Data Structure. Examples of blockchain persisting data structures include *memory variables* provided by the Ethereum blockchains and *private data* provided by the Hyperledger fabric. Both data structures persist across executions of the methods of a smart contract, although both data structures provide different features. We briefly review both before describing how they are used.

Memory variables: Ethereum provides three types of storage that can be used by the methods of a smart contract. They include the *ledger*, *memory variables*, and *stack*. The most expensive is the ledger that the smart contract methods may access using the *Level DB*, which is the previously described key-value data store. Cheapest is the method's stack storage, but with a limit on the size of data that may be stored on the stack. *Memory variables* are also expensive, and they persist across and are accessible by all smart contract methods.

Private data: Hyperledger fabric provides the option of declaring persisting private data that is accessible only to designated actors. In essence, the private data is a separate blockchain that enables reading and writing only to the designated actors.

LtB Transaction Mechanism Implementation. At the risk of redundancy, we review the process of creating an LtB transaction and its supporting mechanism:

1. The developer proceeds with the development of the smart contract, including writing the methods of the LtB transaction, while satisfying the constraints on the transaction methods: The transaction methods must be *independent*. Furthermore, the methods of the LtB transaction must be identified by the developer using special codes that are recognizable by a pre-compiler.
2. The pre-compiler is used to augment the smart contract methods with the patterns that implement the LtB transaction mechanism. The patterns ensure that instead of manipulating the ledger, the transaction methods use the cache. In addition, the pre-compiler inserts the two methods that are used to start and end the transaction, i.e., the *LtB-transaction-begin* and *LtB-transaction-end* methods.

We note that the *atomicity* and *consistency* of the LtB transaction are assured as all ledger writes are made on the cache and that the ledger is updated from the cache by the execution of the *LtB-transaction-end* method. As the ledger is updated by an execution of a single method, the native blockchain ensures the consistency and atomicity of the ledger updates by the *LtB-transaction-end* method. Durability is achieved through the ledger's replication.

However, although updates are made to the cache and not to the ledger itself, updates are written using keys that may be known outside the LtB transaction methods and hence are accessible by non-transaction methods – violating the *isolation* property. To prevent that, the cache needs to perform a translation of the keys, used to write to the ledger, into keys known only to the transaction methods – this translation is maintained by the LtB transaction mechanism.

Following the pre-compilation, the augmented smart contract is ready for compilation and deployment on a blockchain – actions that are automated. It should be noted that a custom pre-compiler is needed for each blockchain supporting LtB transactions.

4 LtB Transaction Mechanism Overhead Cost

In this subsection, we examine the overhead cost of an LtB transaction mechanism. As our assumed environment is that of long-term transactions, comparing the total delays of long-term transactions without and with an LtB transaction mechanism is not appropriate as the overhead delays due to the LtB transaction mechanism would be insignificant in comparison of the long-term delays of the transaction activities itself. Instead, we determine the cost of executing the patterns inserted into the smart contract methods to implement the LtB transaction mechanism. We first discuss our method used to determine the overhead costs and then we discuss evaluation results.

4.1 Cost Estimation Method

To determine the cost of executing patterns that are used to implement the LtB transaction mechanism, we utilize the Ethereum blockchain tools that enable estimation of the cost of execution of smart contract methods written in the Solidity language. Solidity is a Turing-complete, high-language used to write Ethereum smart

contracts. We use Remix to compile smart-contract methods. In addition to producing the object code, the Remix compiler also estimates the gas cost to execute each method.

The cost of a unit of gas fluctuates and depends on the load on the system at any point in time: Increased demand for executing smart contracts by users, means increase in the dollar cost per unit of gas. Thus, the Remix compiler can be used to estimate the gas requirements for a method so that the user would be able to estimate how much gas to allocate for executing a smart contract method. We use the Remix compiler to measure the gas cost of execution of each method of the smart contract with and without the patterns to implement the LtB transaction mechanism and thus estimate the relative overhead cost of providing an LtB transaction mechanism when compared to executing the LtB transaction without an LtB transaction mechanism.

We performed two measurements using a smart contract that contains the following three methods before the smart contract is augmented with patterns, wherein one method is not a part of the transaction and two methods that are a part of an LtB transaction:

- A simple method that is not a part of the transaction. The method writes a string “start” to a memory variable and returns.
- Two methods that are a part of an LtB transaction:
 - Method `m1(x1: objectSizeX) ...` the method writes to the ledger an object of size X bytes.
 - Method `m2()` ... the method reads the object written by the method `m1` and then writes it back to the ledger again as a new object.

We used the Remix compiler to compile the smart contract and calculated the estimated gas cost for the three methods. Thus, the gas-cost is the sum of estimated execution costs of each of the methods for the case without the LtB transaction mechanism. However, the cost for the case when the LtB transaction is implemented, in addition to the cost of the three methods, the cost of the methods to start and end the transaction, the *LtB-transaction-begin* and *LtB-transaction-end* methods, are also included. We used a strip-down version of a pre-compiler, which was simplified to work with the simple example, to replace each read/write to a ledger with a read/write from the cache, i.e., using the Ethereum memory variables. Furthermore, the smart contract was also augmented with the *LtB-transaction-begin* and *LtB-transaction-end* methods, wherein the *LtB-transaction-end* method also replays all the reads and writes stored in the cache to on the ledger itself.

For each of the two cases, the smart contract, written in the Solidity language, was compiled using the Remix compiler that, in addition to the compiled code, also provides the estimate of the gas cost to execute each method. The gas costs for the methods are added up to produce the total gas cost execution for each case, with the gas-cost expressed in gwei units.

4.2 Cost Estimates

Table 1 shows the gas-cost estimates. There are three rows, with the first containing the column headings. Each of the following two rows is for one of the two cases that is identified by the label in the first column: Label *No-LtB* is for the case when there is no LtB transaction mechanism, while the label *With-LtB* is for the case when the smart contract is augmented with patterns implementing the LtB transactions mechanism. The second and subsequent columns are the gas-cost estimates for the size of the object X being $X = 75, 512, 1024$, and 1875 Kb. Note that the 1875 Kb is the maximum data size that one transaction can store in one block of public Ethereum [13]. The table data is shown graphically in Figure 1. We make several observations:

- The overhead cost of the LtB transaction mechanism is not cheap for the Ethereum blockchain.
- The cost is a direct function of the size of the object that is written to and read from the ledger.

Table 1. Gas-cost Estimates.

Label	75 kb	512 kb	1024 kb	1875 kb
No-LtB	4545000	31027200	62054400	113625000
With-LtB	9405000	64204800	124109382	227250582

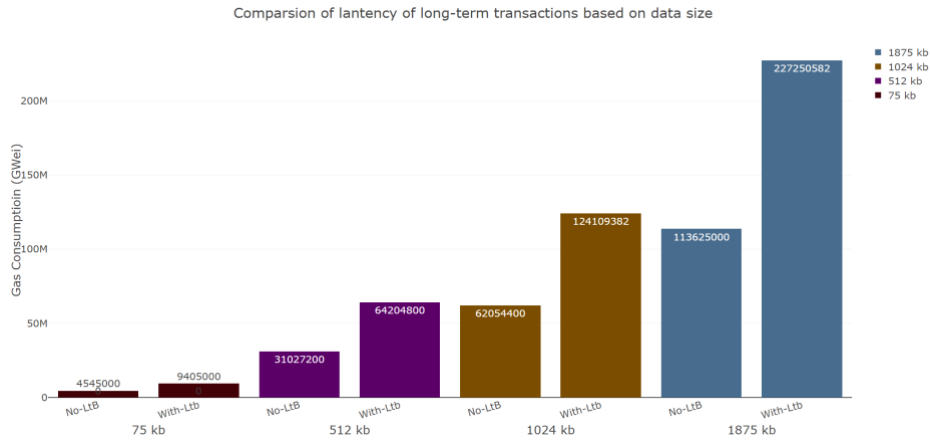


Figure 1: Comparison of Gas-cost Estimates

Furthermore, the overhead cost of the LtB transaction mechanism is not cheap. Even if the transaction methods themselves were minimal in terms of the access to the ledger, the overhead cost includes setting up the cache. However, for permissioned blockchains that have fewer network nodes and for sidechains with a low cost, the cost of replication and the consensus algorithm may be significantly lower and hence the cost of the overhead LtB mechanism may also be lower.

5 Related Work

We showed that in trade of goods and services, there is a need for long-term transaction consisting of multiple methods as transaction activities may be long-term, with unpredictable delays and sequencing of inputs to the transaction by various collaborators. To the best of our knowledge, this problem has is yet to be addressed in the scientific literature. Instead, research and industry concentrated on supporting cross-chain transactions, particularly in support of **Non-fungible Tokens (NFTs)**.

Closest is our previous work on supporting the long-term transactions in blockchains in the context of transforming a BPMN model into the methods of a smart contract [14]. A BPMN model goes through several transformations, eventually resulting in a smart contract(s) that is deployed on a blockchain – all automatically under the guidance of the developer [9]. The BPMN model is extended with a concept of a BPMN long-term transaction that represents the transaction’s collaborative activities by the transaction participants. The BPMN model, with the long-term transaction as represented in an extended BPMN, is analyzed, and it proceeds through several transformations that eventually produce the methods of a smart contract(s) that support the long-term BPMN transaction. Here we addressed the issue of supporting long-term blockchain transactions consisting of multiple methods of a smart contract.

6 Summary, Concluding Remarks, and Future Research

This paper described our initial results on supporting long-term blockchain transactions that span executions of a multiplicity of the methods of a smart contract, wherein such transactions are not supported by blockchains. We provided motivation for supporting the LtB transaction and presented a model of our approach for an LtB transaction mechanism using a generalized roll-up techniques used for blockchains.

We presented an LtB transaction model that includes a set of independent methods and use patterns to implement the LtB transaction mechanism that ensures that the LtB transaction satisfies the ACID transactional properties. Our current implementation uses a data structure that is provided by the blockchain infrastructure and that persists across executions of the methods of a smart contract, such as *memory variables* in Ethereum, or *private data* in Hyperledger fabric. However, our approach may be readily extended to use the mainchain, sidechain or cross-chains, instead of using the blockchain’s persisting data structures. This is important as some blockchains may not support data native data structures that persist across executions of a smart contract methods. Furthermore, using sidechains as layer-2 network may be attractive from the perspective of overhead costs of the LtB mechanism when the sidechain is much less expensive than the mainchain processing.

Although our approach assures that the LtB transaction satisfies the ACID properties, we did not yet tackle the issues of privacy and access control. The issue of privacy arises in blockchains because the ledger data is accessible to all smart contracts. Another issue that arises is access control to execute methods of an LtB

transactions. We are currently working on how to resolve the above issues and nested transaction.

References

1. Khan, S.N., Loukil, F., Ghedira-Guegan, C., Benkhelifa, E., Bani-Hani, A.: Blockchain smart contracts: Applications, challenges, and future trends. *Peer-to-Peer Netw. Appl.* 14, 2901–2925 (2021). <https://doi.org/10.1007/s12083-021-01127-0>.
2. Belchior, R., Vasconcelos, A., Guerreiro, S., Correia, M.: A Survey on Blockchain Interoperability: Past, Present, and Future Trends, <http://arxiv.org/abs/2005.14282>, (2021).
3. Taylor, P.J., Dargahi, T., Dehghantanha, A., Parizi, R.M., Choo, K.-K.R.: A systematic literature review of blockchain cyber security. *Digital Communications and Networks*. 6, 147–156 (2020). <https://doi.org/10.1016/j.dcan.2019.01.005>.
4. Vacca, A., Di Sorbo, A., Visaggio, C.A., Canfora, G.: A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. *Journal of Systems and Software*. 174, 1 (2021). <https://doi.org/10.1016/j.jss.2020.110891>.
5. Yang, D., Long, C., Xu, H., Peng, S.: A Review on Scalability of Blockchain. In: *Proceedings of the 2020 The 2nd Int. Conf. on Blockchain Technology*. pp. 1–6. ACM, New York, NY, USA (2020). <https://doi.org/10.1145/3390566.3391665>.
6. Weber, I., Xu, X., Riveret, R., Governatori, G., Ponomarev, A., Mendling, J.: Untrusted Business Process Monitoring and Execution Using Blockchain. In: La Rosa, M., Loos, P., and Pastor, O. (eds.) *Business Process Management*. pp. 329–347. Springer International Publishing, Cham (2016). https://doi.org/10.1007/978-3-319-45348-4_19.
7. Tran, A.B., Lu, Q., Weber, I.: Lorikeet: A Model-Driven Engineering Tool for Blockchain-Based Business Process Execution and Asset Management.
8. López-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., Ponomarev, A.: CATERPILLAR: A Business Process Execution Engine on the Ethereum Blockchain, <http://arxiv.org/abs/1808.03517>, (2019). <https://doi.org/10.48550/arXiv.1808.03517>.
9. Bodorik, P., Liu, C.G., Jutla, D.: TABS: Transforming automatically BPMN models into blockchain smart contracts. *Elsevier Journal of Blockchain: Research and Applications*. (2022). <https://doi.org/10.1016/j.bcra.2022.100115>.
10. Liu, C., Bodorik, P., Jutla, D.: From BPMN to smart contracts on blockchains: Transforming BPMN to DE-HSM multi-modal model. In: *2021 Int. Conf. on Engineering and Emerging Technologies (ICEET)*. pp. 1–7 (2021). <https://doi.org/10.1109/ICEET53442.2021.9659771>.
11. Object-relational impedance mismatch, https://en.wikipedia.org/w/index.php?title=Object%E2%80%93relational_impedance_mismatch&oldid=1134321907, (2023).
12. HeartBank®: Smart Contract Design Patterns, <https://medium.com/heartbankstudio/smart-contract-design-patterns-8b7ca8b80dfb>, last accessed 2023/01/02.
13. Ethereum Average Block Size, https://ycharts.com/indicators/ethereum_average_block_size, last accessed 2023/01/24.
14. Liu, C.G., Bodorik, P., Jutla, D.: Supporting Long-term Transactions in Smart Contracts. In: *2022 Fourth International Conference on Blockchain Computing and Applications (BCCA)*. pp. 11–19 (2022). <https://doi.org/10.1109/BCCA55292.2022.9922193>.